

Trabajo Práctico 4

Santino Guerreiro, Alexis Peralta, Ignacio Baldassa

Acciones a realizar:

1. Abra su terminal e ingrese a su directorio de trabajo. Cree un archivo de texto con algo de contenido para que ocupe bloques físicos de disco: `echo "Sistemas Operativos - Arquitectura del FS" > arquitectura.txt`.

```
[alexis@ale ~]$ echo "SistemasOperativos - Arquitectura del FS" > arquitectura.txt
```

2. En lugar del clásico `ls`, utilizaremos la herramienta de inspección profunda `stat` para ver la estructura del inodo. Ejecute: `stat arquitectura.txt` (Anote tres datos clave: el número de inodo (Inode), la cantidad de bloques (Blocks) y el contador de enlaces (Links).

```
[alexis@ale ~]$ stat arquitectura.txt
Fichero: arquitectura.txt
  Tamaño: 41          Bloques: 8          Bloque E/S: 4096   fichero regular
Device: 179,2    Inode: 134979      Links: 1
Acceso: (0644/-rw-r--r--)  Uid: ( 1000/  alexis)   Gid: ( 1000/  alexis)
  Acceso: 2026-06-08 09:44:08.687492863 -0300
Modificación: 2026-06-08 09:44:08.687492863 -0300
    Cambio: 2026-06-08 09:44:08.687492863 -0300
  Creación: 2026-06-08 09:44:08.687492863 -0300
[alexis@ale ~]$ ln arquitectura.txt arquitectu.txt
```

3. Creación del Enlace Duro (Hard Link): Cree un enlace duro del archivo original `arquitectura.txt`.

```
  Creación: 2026-06-08 09:44:08.687492863 -0300
[alexis@ale ~]$ ln arquitectura.txt arquitectu.txt
```

4. Ejecute nuevamente `stat` del archivo original y del enlace duro. Compare las salidas.

```
[alexis@ale ~]$ stat arquitectura.txt
Fichero: arquitectura.txt
  Tamaño: 41          Bloques: 8          Bloque E/S: 4096   fichero regular
Device: 179,2    Inode: 134979      Links: 2
Acceso: (0644/-rw-r--r--)  Uid: ( 1000/  alexis)   Gid: ( 1000/  alexis)
  Acceso: 2026-06-08 09:44:08.687492863 -0300
Modificación: 2026-06-08 09:44:08.687492863 -0300
    Cambio: 2026-06-08 09:54:16.743662797 -0300
  Creación: 2026-06-08 09:44:08.687492863 -0300
[alexis@ale ~]$ stat arquitectu.txt
Fichero: arquitectu.txt
  Tamaño: 41          Bloques: 8          Bloque E/S: 4096   fichero regular
Device: 179,2    Inode: 134979      Links: 2
Acceso: (0644/-rw-r--r--)  Uid: ( 1000/  alexis)   Gid: ( 1000/  alexis)
  Acceso: 2026-06-08 09:44:08.687492863 -0300
Modificación: 2026-06-08 09:44:08.687492863 -0300
    Cambio: 2026-06-08 09:54:16.743662797 -0300
  Creación: 2026-06-08 09:44:08.687492863 -0300
```

5. Creación del Enlace Simbólico (Soft Link): Cree un enlace simbolico(soft) del archivo original `arquitectura.txt`.

```
[alexis@ale ~]$ ln -s arquitectura.txt arquitectuS.txt
[alexis@ale ~]$ stat arquitectuS.txt
```

6. Analice la estructura del nuevo enlace con el comando `stat` (Observe el tamaño del archivo - Size - y compárelo con la cantidad de caracteres de la palabra

"arquitectura.txt").

```
[alexis@ale ~]$ stat arquS.txt
  Fichero: arquS.txt -> arquitectura.txt
   Tamaño: 16          Bloques: 0          Bloque E/S: 4096   enlace simbólico
Device: 179,2   Inode: 135128      Links: 1
Acceso: (0777/lrwxrwxrwx)  Uid: ( 1000/ alexis)   Gid: ( 1000/ alexis)
  Acceso: 2026-06-08 10:01:29.982380352 -0300
Modificación: 2026-06-08 10:01:29.982380352 -0300
  Cambio: 2026-06-08 10:01:29.982380352 -0300
 Creación: 2026-06-08 10:01:29.982380352 -0300
```

7. Simulación de inconsistencia (Puntero colgante): Elimine la entrada de directorio original: `arquitectura.txt`.

```
[alexis@ale ~]$ rm -r ~/arquitectura.txt
```

8. Intente leer ambos enlaces con el comando `cat`.

```
[alexis@ale ~]$ rm -r ~/arquitectura.txt
[alexis@ale ~]$ cat arqu.txt
SistemasOperativos - Arquitectura del FS
[alexis@ale ~]$ cat arquS.txt
cat: arquS.txt: No existe el fichero o el directorio
```

Preguntas de Análisis:

1. **Gestión del contador de referencias:** En el paso 4, ¿qué valor mostró el campo Links (contador de enlaces) en el inodo? ¿Cómo utiliza el Sistema Operativo este contador para saber cuándo debe liberar los bloques de memoria física del disco tras usar el comando `rm`?

Tras la ejecución de `stat` ambos archivos muestran el mismo número (2) en el contador de enlaces y el mismo inodo (134979). Esto implica que, el Hard Link suma un Link, a diferencia del Soft Link, por esto si el archivo tiene un Hard Link se creará otro Link, el sistema operativo solo borrara los datos si los links son 0, digamos, si no hay acceso posible a esa información.

2. **Diferenciación de Inodos:** Al analizar el enlace simbólico en el paso 6, ¿por qué su número de inodo y tamaño difieren del archivo original? ¿Qué está almacenando exactamente el SO en los bloques de datos de este enlace?

Cuando se utiliza `stat` en el Soft Link su número de inodo y tamaño son distintos del original (contraste al Hard Link, donde son iguales) porque apunta a la ruta del archivo, no a la dirección física en disco. El Soft Link es un archivo nuevo con un atajo, en los bloques de datos se guarda esta ruta que actúa como un puntero.

3. **Persistencia de los datos:** En el paso 8, tras eliminar `arquitectura.txt`, usted aún pudo leer el contenido mediante `link_duro.txt`. A nivel del sistema de archivos, explique detalladamente por qué los datos no se perdieron.

Los datos están directamente relacionados al Hard Link, con lo cual, este no se perdió. A diferencia del Soft Link, el cual solo referencia al archivo original, que al ser borrado hizo que el Soft Link no tuviera más acceso a los datos.

4. **Limitaciones de la VFS (Virtual File System):** Investigue en la bibliografía de la materia: ¿Por qué el kernel de Linux impide rotundamente la creación de un Hard Link si el destino se encuentra en una partición de disco o sistema de archivos diferente, pero sí permite que un Soft Link cruce esas fronteras?

Debido a que los valores de los inodos son independientes en cada partición no es posible realizar un Hard Link desde otra partición debido a que causaría conflictos, sin embargo los Soft Link reciben un valor inode único y diferente al Hard Link, los Hard Link contienen el mismo inodo.